

# Software Supply Chain Attacks and Build Provenance

## 軟體供應鏈攻擊與建置來源證明之研究報告

NG, Chun Sing (r14922186)      OU, Chia-Yun (r14922072)

2026 年 5 月 23 日

### 摘要

現代軟體開發高度依賴開源套件、套件管理器、持續整合與持續部署流程，以及第三方套件註冊庫。這些機制提升了開發效率，卻也形成軟體供應鏈攻擊（software supply chain attacks）的主要攻擊面。攻擊者不一定需要直接攻擊最終應用程式，而是可以透過竊取維護者帳號、污染依賴套件、操控建置快取、入侵 CI/CD workflow 或發布惡意套件版本，使惡意程式碼沿著信任鏈傳遞到下游使用者。

本報告以 npm 生態系為核心案例，探討軟體供應鏈攻擊的風險、近期事件、常見防禦方法，以及 build provenance 的角色。本文首先說明軟體供應鏈安全為何成為 OWASP Top 10:2025 的重要風險類別，接著分析 Axios npm compromise 與 TanStack npm supply-chain compromise 兩個案例。Axios 事件呈現維護者帳號遭竊後，攻擊者如何發布惡意版本並利用 installation script 安裝 RAT；TanStack 事件則揭示更深層的限制：即使套件具備合法的 build provenance，若 CI/CD workflow 本身已遭污染，provenance 仍可能只是誠實地證明「惡意產物確實來自被污染的合法流程」。

本文主張，build provenance 是必要但不足夠的防禦。它能提升 artifact 的可追溯性與防竄改能力，但不能單獨證明 artifact 無害。有效的軟體供應鏈安全應採取 layered defense：同時結合 dependency scanning、lockfile discipline、install-time verification、maintainer account security、CI/CD isolation、cache hardening、credential restriction 與 provenance verification。

**關鍵字：**軟體供應鏈安全、build provenance、SLSA、Sigstore、npm、CI/CD security、dependency attack

# 目錄

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>1 Introduction</b>                                        | <b>3</b>  |
| <b>2 Background: Software Supply Chain Security</b>          | <b>3</b>  |
| 2.1 Software Supply Chain 的定義 . . . . .                      | 3         |
| 2.2 為何此問題成為主要風險 . . . . .                                    | 4         |
| <b>3 Threat Model and Attack Surfaces</b>                    | <b>4</b>  |
| <b>4 Case Study I: Axios npm Compromise</b>                  | <b>5</b>  |
| 4.1 事件概述 . . . . .                                           | 5         |
| 4.2 攻擊流程 . . . . .                                           | 5         |
| 4.3 此案例揭示的防禦缺口 . . . . .                                     | 5         |
| <b>5 Build Provenance as a Defense Mechanism</b>             | <b>6</b>  |
| 5.1 Build Provenance 的概念 . . . . .                           | 6         |
| 5.2 Producer 與 Consumer Workflow . . . . .                   | 6         |
| 5.3 Technology Stack . . . . .                               | 7         |
| <b>6 Case Study II: TanStack npm Supply-Chain Compromise</b> | <b>7</b>  |
| 6.1 事件概述 . . . . .                                           | 7         |
| 6.2 為什麼這對 provenance 是嚴重挑戰 . . . . .                         | 8         |
| 6.3 CI/CD 層面的風險 . . . . .                                    | 8         |
| <b>7 Comparison of Defense Mechanisms</b>                    | <b>9</b>  |
| <b>8 Toward a Layered Defense Strategy</b>                   | <b>9</b>  |
| 8.1 Dependency Layer . . . . .                               | 9         |
| 8.2 Source Layer . . . . .                                   | 10        |
| 8.3 Build Layer . . . . .                                    | 10        |
| 8.4 Release Layer . . . . .                                  | 10        |
| <b>9 Discussion</b>                                          | <b>11</b> |
| 9.1 Build Provenance 的實際價值 . . . . .                         | 11        |
| 9.2 Build Provenance 的根本限制 . . . . .                         | 11        |
| 9.3 研究與實作方向 . . . . .                                        | 11        |
| <b>10 Conclusion</b>                                         | <b>12</b> |

# 1 Introduction

現代軟體開發並不是一個完全由開發者自行控制的封閉流程。開發者每天都可能執行下列指令：

```
npm install  
pip install  
mvn install  
conda install
```

這些指令會自動下載第三方程式碼、解析 **transitive dependencies**，並在某些套件生態系中執行 **installation lifecycle scripts**。從效率角度來看，這使開發者能快速重用既有軟體元件；但從安全角度來看，這也代表開發者將相當大的信任交給套件維護者、套件註冊庫、CI/CD pipeline、build cache、release automation 以及各種中介工具。

軟體供應鏈攻擊的核心危險在於，攻擊者不需要直接突破最終應用程式的防線，而是可以攻擊上游元件或中介流程。一旦攻擊者成功污染一個廣泛使用的 **dependency**，惡意程式碼就可能被大量下游專案自動下載、建置或執行。這種攻擊模式具有高度擴散性，也使傳統以單一應用程式為邊界的安全思維變得不足。

本報告聚焦於一個研究問題：**build provenance** 是否能有效防止軟體供應鏈攻擊？本文的答案是：**build provenance** 能提高攻擊成本並強化 **artifact** 的可驗證性，但它不是完整解法。若 CI/CD pipeline、build cache 或 workflow 權限本身遭到污染，**provenance** 仍可能產生看似合法但實際上危險的證明。因此，**provenance** 必須與更完整的供應鏈防禦策略搭配使用。

## 2 Background: Software Supply Chain Security

### 2.1 Software Supply Chain 的定義

根據 NIST 的定義，**software supply chain** 是一系列用來建立、轉換、評估軟體 **artifact** 品質與政策符合性的步驟。實務上，**software supply chain** 不只包含原始碼，也包含第三方 **dependencies**、版本控制系統、build tools、CI/CD pipelines、package registries、release workflows 與部署機制。

因此，軟體供應鏈攻擊可以被理解為：攻擊者入侵或操控供應鏈中某個原本被信任的節點，進而讓惡意程式碼或被竄改的 **artifact** 透過正常流程進入下游系統。被攻擊的節點可能是 **dependency**、**maintainer account**、**build cache**、**CI/CD workflow**、**artifact registry** 或 **signing mechanism**。

## 2.2 為何此問題成為主要風險

OWASP Top 10:2025 將 Software Supply Chain Failures 列為 A03，反映現代應用程式安全風險的轉移。過去常見的應用程式安全模型多聚焦於應用程式本身的程式碼缺陷，例如 injection、broken authentication 或 access control failure。然而，現代軟體的攻擊面已經外擴到「應用程式如何被建置、由誰發布、依賴哪些元件、如何取得 artifact」等問題。

這個風險上升的原因可整理如下：

- (1) **Dependency 數量龐大**：一個專案可能只宣告少數 direct dependencies，但實際上會引入大量 transitive dependencies。
- (2) **安裝流程高度自動化**：套件管理器會自動解析版本、下載套件、檢查 metadata，甚至執行 lifecycle scripts。
- (3) **CI/CD 成為發布核心**：許多套件由 GitHub Actions 或其他 CI/CD 系統自動建置與發布，使 workflow 權限與 runner 環境成為重要攻擊面。
- (4) **信任鏈難以人工審查**：開發者通常不會逐一檢查每個套件版本、tarball 內容、build log 與 artifact hash。
- (5) **單點妥協可大規模擴散**：熱門套件一旦遭攻擊，影響範圍可能橫跨眾多組織與下游產品。

## 3 Threat Model and Attack Surfaces

軟體供應鏈攻擊的攻擊面可分為 dependency layer、source layer、build layer 與 release layer。不同攻擊面對應不同的防禦需求。

表 1: 軟體供應鏈中的主要攻擊面

| Layer      | Possible Attack Surface                                                                              |
|------------|------------------------------------------------------------------------------------------------------|
| Dependency | 惡意套件、typosquatting、dependency confusion、transitive dependency 污染、postinstall script 濫用               |
| Source     | 維護者帳號遭竊、惡意 commit、review bypass、protected branch 設定不當                                                |
| Build      | CI/CD workflow 被竄改、build cache poisoning、runner memory token extraction、unsafe pull request workflow |
| Release    | registry account compromise、artifact substitution、缺乏 signature、缺乏 provenance verification            |

在 npm 生態系中，lifecycle scripts 是特別值得關注的攻擊點。例如 'postinstall' script 可在開發者安裝套件時自動執行。如果攻擊者成功發布帶有惡意 lifecy-

cle script 的版本，即使使用者沒有主動執行該套件的程式碼，也可能在安裝階段就已經被攻擊。

## 4 Case Study I: Axios npm Compromise

### 4.1 事件概述

Axios 是 JavaScript 生態系中廣泛使用的 HTTP client 套件。根據簡報內容，Axios 在 2026 年 3 月 31 日發生 npm supply chain compromise。攻擊者透過遭竊的 maintainer credentials 發布惡意版本，使安裝 Axios 的使用者可能被導向惡意依賴套件，並在安裝過程中執行 postinstall script，最終安裝 remote access trojan (RAT)。

此事件具有高度代表性，因為它呈現了 open-source package ecosystem 的典型風險：當熱門套件的發布權限遭到濫用時，下游使用者可能難以及時察覺。套件名稱、registry 頁面、版本流程都可能看起來正常，但實際 tarball 已經被攻擊者污染。

### 4.2 攻擊流程

Axios 事件可概念化為下列流程：

```
npm install axios
-> malicious axios version
    -> attacker-controlled dependency
        -> postinstall script executes
            -> RAT installed silently
```

此流程的危險之處在於它符合開發者日常操作。使用者可能只是執行一般的 'npm install'，卻在不知情的情況下觸發惡意 installation script。也就是說，攻擊不是發生在應用程式 runtime，而是在 dependency installation 階段就已經完成。

### 4.3 此案例揭示的防禦缺口

Axios 事件顯示，傳統防禦工具存在幾個限制：

- (1) **CVE-based scanner** 的時間落差：像 'npm audit'、Snyk 等工具對已知 CVE 有效，但新發布的惡意版本可能尚未被登錄為 CVE。
- (2) **lockfile** 不是絕對防線：lockfile 可以固定版本與 hash，但若 lockfile 是在惡意版本存在期間產生，它反而會固定被污染的版本。

- (3) 自動更新工具可能引入風險：Dependabot 類工具能協助更新 vulnerable dependencies，但如果最新版本本身是惡意版本，自動更新也可能擴大風險。
- (4) 使用者缺乏 **install-time visibility**：一般安裝流程通常不會要求使用者審查 package provenance、publisher identity 或 lifecycle script 行為。

因此，Axios 事件支持一個重要結論：軟體供應鏈防禦不能只依賴 known vulnerability scanning，也必須檢查 package metadata、發布行為、維護者信任狀態與安裝時風險。

## 5 Build Provenance as a Defense Mechanism

### 5.1 Build Provenance 的概念

Build provenance 是描述 artifact 如何被產生的 metadata。它通常記錄 source repository、commit hash、builder identity、build workflow、build time、artifact hash 等資訊。其核心目標是讓使用者或自動化工具能回答下列問題：

- 這個 artifact 是否由預期的 source commit 建置？
- 它是否由預期的 CI/CD workflow 產生？
- build steps 是否符合預期？
- registry 中的 tarball 是否與 CI output 相符？

在 SLSA 與 Sigstore 的模型中，build provenance 可以透過 keyless signing、OIDC identity、artifact hash 與 transparency log 等機制進行驗證。其理想效果是使 artifact 從 source 到 release 的過程變得 tamper-evident。

### 5.2 Producer 與 Consumer Workflow

一個 provenance-based workflow 可分為 producer 與 consumer 兩端。

#### **Producer side：**

- (1) CI/CD 系統從 source repository 中指定 commit 建置 artifact。
- (2) 建置流程產生 provenance statement，記錄 commit、builder、build type 與 artifact hash。
- (3) Provenance 透過 Sigstore 等機制簽章，並記錄至 transparency log。
- (4) Registry 將 package 與 attestation metadata 一起保存。

**Consumer side :**

- (1) 安裝前取得 package 與對應 attestation。
- (2) 驗證 provenance signature。
- (3) 比對 source repository、builder identity、build type 與 expected policy 是否相符。
- (4) 若 package 過去具有 provenance 但新版本突然缺少 provenance，可提出 warning 或直接阻擋。

此模式將 provenance 從「記錄」轉化為「預防」。換言之，provenance 本身只是 metadata；真正的安全效果來自 install-time verification 與 policy enforcement。

## 5.3 Technology Stack

表 2: Build provenance 相關技術層次

| Layer        | Technology            | Role                                  |
|--------------|-----------------------|---------------------------------------|
| Framework    | SLSA                  | 定義 provenance 應證明的內容與 build level     |
| Cryptography | Sigstore              | 提供 keyless signing 與 transparency log |
| Platform     | Registry + CI         | 產生、儲存與發布 attestation                  |
| Consumer     | npq / Socket Firewall | 在安裝前驗證 provenance 或套件風險               |

Build provenance 的最大價值在於可追溯性。當 artifact 被發布到 registry 時，使用者不再只能信任 package name 與 version number，而可以要求 artifact 提供其來源與建置證明。

## 6 Case Study II: TanStack npm Supply-Chain Compromise

### 6.1 事件概述

TanStack 事件顯示 build provenance 的限制。根據簡報內容，TanStack 在 2026 年 5 月 11 日遭遇 npm supply-chain compromise。攻擊者在短時間內影

響 42 個 packages，發布 84 個 malicious versions，payload 目標包括 cloud credentials、API tokens 與 SSH keys。

與 Axios 事件不同，TanStack 的關鍵不是單純盜用 npm maintainer account 後直接發布套件，而是攻擊 CI/CD workflow 與 cache trust boundary。攻擊者利用 fork pull request 與 workflow 行為，污染 shared CI cache。當合法 release workflow 執行時，它恢復了被污染的 cache，進而在合法流程中產生與發布惡意套件。

## 6.2 為什麼這對 provenance 是嚴重挑戰

TanStack 事件最重要的意義在於：惡意版本具有 valid build provenance。也就是說，provenance 正確地記錄了 artifact 來自 TanStack 的 workflow。然而，如果該 workflow 的執行環境已經被 cache poisoning 或 token extraction 操控，provenance 仍然可能看起來正常。

這導出一個核心觀點：

**Provenance proves origin, not innocence.**

Provenance 可以證明 artifact 來自哪個 repository、哪個 builder、哪個 workflow，但不能單獨證明該 workflow 在執行時沒有被污染。換言之，provenance 是供應鏈安全的必要條件，但不是充分條件。

## 6.3 CI/CD 層面的風險

TanStack 事件使下列 CI/CD 風險更加明顯：

- (1) **Cache poisoning**：若 untrusted workflow 與 trusted release workflow 共用 cache，攻擊者可能透過 cache 注入惡意內容。
- (2) **OIDC token exposure**：CI runner 中的 token 若能被惡意程式碼讀取，攻擊者可能取得發布權限或偽造合法流程。
- (3) **Workflow trust boundary 不清**：fork PR、pull\_request\_target 與 release workflow 若權限分離不足，會形成高風險組合。
- (4) **合法 pipeline 被濫用**：當攻擊者不需要繞過 pipeline，而是使 pipeline 自己建置惡意 artifact 時，傳統 provenance 檢查會失效。

因此，build provenance 必須搭配 CI/CD hardening。否則 provenance 可能只是提供一份精確但不充分的紀錄。



## 7 Comparison of Defense Mechanisms

表 3: 常見供應鏈防禦機制比較

| Defense                   | Protects Against                                                       | Limitations                                        |
|---------------------------|------------------------------------------------------------------------|----------------------------------------------------|
| 'npm audit' / Snyk        | 已知 CVE 與已登錄 vulnerable dependencies                                    | 對新型 malicious package、未登錄 CVE 或 registry 行為異常不一定有效 |
| Lockfile                  | 固定版本與 hash，提升 reproducibility                                          | 若 lockfile 產生時已包含惡意版本，lockfile 會固定該風險              |
| Dependabot                | 自動更新已知 vulnerable dependencies                                         | 新版本本身可能是惡意版本；仍依賴 vulnerability database            |
| Build provenance          | 確認 artifact 來源、builder identity、artifact hash 與 build process metadata | 無法單獨保證 builder 或 CI/CD workflow 未被污染               |
| Install-time verification | 在安裝前執行 provenance 或 package risk policy                                | 需要維護 expected policy，也可能受到 adoption rate 限制        |
| CI/CD hardening           | 降低 workflow、cache、token、runner 被濫用的機率                                  | 需要工程實作成本；不同平台與專案設定差異大                              |

從此比較可見，沒有單一防禦能涵蓋所有供應鏈攻擊。'npm audit' 對 known vulnerabilities 有用，但對 zero-day malicious package 不夠。Lockfile 對 reproducibility 有用，但不一定能判斷 package 是否惡意。Build provenance 能強化來源可驗證性，但不能替代 CI/CD 環境的安全性。因此，最合理的方向是 defense in depth。

## 8 Toward a Layered Defense Strategy

根據上述案例與分析，實務上的軟體供應鏈防禦應至少涵蓋四個層次。

### 8.1 Dependency Layer

Dependency layer 的目標是降低惡意或脆弱套件進入專案的機率。可採取的策略包括：

- 使用 lockfile 並避免隨意更新 dependency。
- 使用 SBOM 記錄專案依賴組成。
- 整合 'npm audit'、OSV、Snyk 等 vulnerability scanners。
- 使用 npq、Socket Firewall 等 install-time gate 檢查 package metadata、provenance 與 suspicious behavior。
- 對剛發布的新版本設定 minimum release age，避免立即安裝極新的高風險版本。

## 8.2 Source Layer

Source layer 的重點是確保 source repository 與 maintainer account 不容易被濫用。可採取的策略包括：

- 要求 maintainer 使用 MFA。
- 使用 protected branches 與 mandatory code review。
- 限制 package publish 權限，避免單一帳號擁有過大發布權。
- 監控異常 commit、release 或 token 使用行為。

## 8.3 Build Layer

Build layer 是 TanStack 事件揭示的核心問題。可採取的策略包括：

- 將 untrusted PR workflow 與 trusted release workflow 嚴格隔離。
- 避免 fork PR 與 release workflow 共用高信任 cache。
- 最小化 GitHub Actions token、OIDC token 與 registry token 權限。
- 避免在 runner memory 或 log 中暴露 credential materials。
- 對 cache restore path、build script 與 release script 進行更嚴格的審查。

## 8.4 Release Layer

Release layer 的目標是讓 artifact 在發布與安裝時具備可驗證性。可採取的策略包括：

- 使用 build provenance 與 signed attestations。

- 將 artifact hash、builder identity 與 source commit 記錄於 transparency log。
- 在安裝時驗證 provenance，而非只在發布時產生 provenance。
- 若 package 過去具有 provenance，但新版本突然缺少 provenance，應提出警告或阻擋。

## 9 Discussion

### 9.1 Build Provenance 的實際價值

Build provenance 的價值不應被低估。它解決了傳統 package ecosystem 的一個重要問題：使用者過去很難知道 registry 上的 artifact 是否真的來自對應 source code 與合法 build process。Provenance 能將 source、builder 與 artifact 之間建立可驗證關係，並使 artifact substitution、未授權發布與部分 maintainer compromise 更容易被偵測。

此外，provenance 也使 policy enforcement 成為可能。例如 organization 可以要求只安裝具有有效 provenance 的 package，或只信任特定 repository、builder 與 workflow 產出的 artifact。這使供應鏈安全從人工信任轉向機器可驗證的政策。

### 9.2 Build Provenance 的根本限制

然而，build provenance 的限制也必須被清楚理解。Provenance 不是 malware scanner，也不是對 build environment 的完整安全證明。若合法 workflow 被攻擊者控制或污染，provenance 仍會記錄該 workflow 產生的 artifact。這就是 TanStack 事件的重要教訓。

因此，provenance 的正確定位應是：

Build provenance can make the supply chain tamper-evident, but it cannot make a compromised build trustworthy.

這表示 build provenance 必須與 CI/CD hardening、credential isolation、cache control 與 install-time behavioral checks 搭配，才可能形成完整防線。

### 9.3 研究與實作方向

若要將本主題發展為實作型專案，可以設計一個 npm supply chain scanner，整合下列功能：

- (1) 解析 'package.json' 與 lockfile，建立 dependency graph。
- (2) 使用 'npm install --package-lock-only --ignore-scripts --audit=false' 產生或更新 lockfile，但避免執行 lifecycle scripts。
- (3) 整合 'npm audit' 與 OSV，檢查 known vulnerabilities。
- (4) 整合 npq，以 install-time risk checker 的方式檢查 package metadata 與 provenance 狀態。
- (5) 檢查 package 是否包含 lifecycle scripts，例如 'preinstall'、'install'、'postinstall'。
- (6) 輸出 HTML 或 JSON report，呈現 vulnerability counts、suspicious packages、provenance status 與 risk score。

這樣的實作可以清楚展示不同工具的互補性：'npm audit' 與 OSV 偏向 known vulnerabilities，npq 偏向 install-time package risk，而自訂 scanner 可補上 lifecycle script、metadata anomaly 與 reporting layer。

## 10 Conclusion

軟體供應鏈攻擊之所以危險，是因為它利用了現代軟體開發不可避免的信任關係。開發者信任套件管理器、registry、dependency maintainer、CI/CD workflow 與自動發布流程；攻擊者則透過污染其中任一環節，讓惡意程式碼以合法外觀進入下游系統。

Axios 事件說明 maintainer account compromise 與 malicious package publication 可以快速影響大量使用者。TanStack 事件則進一步說明，即使導入 build provenance，若 CI/CD workflow 或 build cache 被污染，provenance 仍可能無法阻止攻擊。這兩個案例共同指出：供應鏈安全不應只依賴單一工具或單一概念。

Build provenance 是軟體供應鏈安全的重要進展。它能回答 artifact 是否來自預期 source、builder 與 build process，並提供 tamper-evident 的驗證基礎。然而，它不能單獨證明 artifact 無害。真正有效的防禦應採取 layered defense，包含 dependency scanning、lockfile discipline、install-time verification、maintainer account protection、CI/CD isolation、cache hardening、credential restriction 與 provenance verification。

總結而言，build provenance 的正確價值不是「保證安全」，而是「讓供應鏈中的信任關係變得可檢查」。在未來的供應鏈安全實務中，provenance 應作為基礎機制之一，並與更嚴格的 build environment security 與 runtime policy enforcement 一起部署。

## 參考文獻

- [1] National Institute of Standards and Technology. *NIST SP 800-204D: Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204D.pdf>
- [2] OWASP. *Software Supply Chain Security Cheat Sheet*. [https://cheatsheetseries.owasp.org/cheatsheets/Software\\_Supply\\_Chain\\_Security\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Software_Supply_Chain_Security_Cheat_Sheet.html)
- [3] OWASP. *A03:2025 Software Supply Chain Failures*. [https://owasp.org/Top10/2025/A03\\_2025-Software\\_Supply\\_Chain\\_Failures/](https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/)
- [4] OWASP. *OWASP Top 10:2025 Introduction and Methodology*. [https://owasp.org/Top10/2025/0x00\\_2025-Introduction/](https://owasp.org/Top10/2025/0x00_2025-Introduction/)
- [5] SLSA. *Supply Chain Threats*. <https://slsa.dev/spec/v1.0/threats-overview>
- [6] SLSA. *About SLSA*. <https://slsa.dev/spec/v1.2/about>
- [7] SLSA. *Verifying Artifacts*. <https://slsa.dev/spec/v1.0/verifying-artifacts>
- [8] OpenSSF. *Sigstore*. <https://www.sigstore.dev/>
- [9] Liran Tal. *npq: Safely install packages with npm or yarn by auditing them as part of your install process*. <https://github.com/lirantal/npq>
- [10] Socket. *Introducing Socket Firewall*. <https://socket.dev/blog/introducing-socket-firewall>
- [11] Axios. *Post Mortem: axios npm supply chain compromise*. <https://github.com/axios/axios/issues/10636>
- [12] Microsoft Security Blog. *Mitigating the Axios npm supply chain compromise*. <https://www.microsoft.com/en-us/security/blog/2026/04/01/mitigating-the-axios-npm-supply-chain-compromise/>
- [13] TanStack. *Postmortem: TanStack npm supply-chain compromise*. <https://tanstack.com/blog/npm-supply-chain-compromise-postmortem>

- [14] TanStack. *Hardening TanStack After the npm Compromise*. <https://tanstack.com/blog/incident-followup>
- [15] GitHub Blog. *Artifact Attestations is generally available*. <https://github.blog/changelog/2024-06-25-artifact-attestations-is-generally-available/>
- [16] GitHub Blog. *Introducing npm package provenance*. <https://github.blog/security/supply-chain-security/introducing-npm-package-provenance/>
- [17] PyPI Blog. *PyPI now supports digital attestations*. <https://blog.pypi.org/posts/2024-11-14-pypi-now-supports-digital-attestations/>
- [18] ReversingLabs. *Software Supply Chain Security Report 2026*. <https://www.reversinglabs.com/resources/software-supply-chain-security-report-2026>